Computers & Security 158 (2025) 104641

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full Length Article

VERTFuzz: Version transformer-driven fuzzing for complex file parsers

Zhaoyu Wen [1,*], Zhiqiang Wang [1], Biao Liu

Beijing Electronic Science and Technology Institute, Beijing, China

ARTICLE INFO

Keywords:
Fuzzing
Version transformer
Vulnerability detection
Automated test generation

ABSTRACT

Fuzzing test technology has seen significant growth in recent years and has evolved into an important tool for more thoroughly and efficiently identifying programme vulnerabilities and defects. However, fuzzing test for complex format files remains challenging. Most fuzz testers require extensive expert knowledge and heavily rely on manually constructed format models, or struggle to accurately identify complex structural relationships, resulting in numerous invalid test variants. In this paper, we propose a metadata-based mutation technique that leverages deep learning models to identify metadata location information and incorporate it into specific mutations, enabling rapid identification of file structures. We also utilise the Version Transformer model to filter out valid test cases from the queue, effectively addressing the issue of sparse defect space in input, making the mutated test cases more effective. Experimental results show that VERTFuzz has identified 32 unique errors across ten different programs, including four complex file formats. On average, VERTFuzz discovered 29% more paths and 14.54% more code blocks than AFL++.

1. Introduction

Fuzzing is a software testing technique that involves providing a program with large volumes of random or intentionally mutated data to uncover potential vulnerabilities and defects. Fuzzing is widely applied in vulnerability discovery, patching, and security assessment, effectively identifying security issues such as memory overflows and unhandled exceptions.

In recent years, various efficient fuzzers have emerged, such as AFL, HonggFuzz, and OSS-Fuzz (Zalewski et al., 2020; robertswiecki and Bechtsoudis, 2024; Serebryany, 2017). However, when it comes to fuzzing techniques for complex file formats, generating compliant seed files and performing mutations on such formats still present significant challenges. Complex file formats often adhere to specific format specifications, and if the input data deviates from these requirements, the target program may refuse to parse it, hindering the ability to conduct deep fuzzing. For example, PDF files feature a nested structure containing multiple levels of objects, compressed data streams, and metadata. Direct mutations in such files can prevent access to deeper program logic.

In response to the above challenges, researchers have proposed various fuzzing test solutions for complex structured files. Using the generated fuzz tester Peach (Peach Teach 2020), which employs predefined input format models, high-quality test cases can be generated in a structured manner, thereby enhancing the effectiveness and syntactic validity of testing to some extent. AFLSmart (Pham et al., 2021), on the other hand, introduces advanced structural representations of files, elevating fuzzing test operations from the byte level to the semantic level. By leveraging structural mutation operations, it more efficiently explores the input space. Furthermore, WEIZZ (A. Fioraldi et al., 2020) proposes an automatic structure-aware mutation technique that does not require prior format information for unknown or undocumented binary formats. It infers block boundaries through dynamic dependency analysis and achieves effective mutation while maintaining a certain degree of syntactic validity.

Although the above methods have achieved significant breakthroughs in structure-aware fuzzing test, they still face several key issues in terms of practicality and adaptability. First, Peach generated fuzz testers heavily rely on manually constructed format models, which not only require significant involvement from domain experts but also result in the testers lacking robustness and adaptability when faced with format updates or variant file structures. Second, such methods often strictly enforce input compliance with specifications, potentially overlooking the potential of over-specification inputs to trigger abnormal behaviour or even vulnerabilities in real-world applications, leading to conservative testing. In contrast, structure-aware fuzz testers like

* Corresponding author.
E-mail address: 2473648203@qq.com (Z. Wen).
[1] These authors contributed equally to this work.

https://doi.org/10.1016/j.cose.2025.104641
Received 10 March 2025; Received in revised form 16 June 2025; Accepted 20 August 2025
Available online 20 August 2025

AFLSmart and WEIZZ reduce reliance on format models but still struggle to accurately identify certain complex or nested structures. Additionally, they may generate invalid mutations when encountering format ambiguity boundaries or implicit syntax dependencies.

To address these challenges, this paper presents VERTFuzz, a mutation-based grey-box fuzzer that utilizes 010 Editor (SweetSweet and raeme, 2024) scripts to extract the format from templates and use it as input for VERTFuzz. Subsequently, in order to eliminate the need to write templates, we designed a deep learning model to automate the tagging of metadata based on the BERT model, which we named ByteBERT. ByteBERT utilizes the data identified by the 010 Editor as a training set for training, and quickly tags untrained file formats by fine-tuning them. We apply the marked format during mutation, similar to the effector map mechanism in AFL. By reading a format file equivalent in size to the input file, it participates in the Havoc mutation mechanism. In each cycle, we maintain reusable space to store the index of each format label corresponding to the file. We replace the original random functions such that every mutation corresponds to a position marked by the format. This approach is based on our testing experience and intuition that, compared to mutating the content of files, mutating the metadata is more likely to trigger new paths. It also makes it easier to detect errors caused by inputs that deviate from the specification. However, this introduces a new problem: if structural mutations are applied in a brute-force manner, the results may be less effective than those of the aforementioned fuzzers, as VERTFuzz might fail to meet the format requirements of the file.

To address the aforementioned shortcomings, we introduced the Version Transformer (VIT) model (Dosovitskiy et al., 2025) to filter out mutated test cases. The core innovation of the VIT model lies in applying the Transformer architecture to image processing. Unlike traditional CNNs, VIT divides images into fixed-size patches and linearly embeds these patches as sequences, which are then processed through a Transformer encoder. This design enables VIT to capture global dependencies within images, rather than just local features. The VIT model is fully compatible with bitmap files used in fuzzing test. Each byte in a bitmap file represents a path within the program, and each byte is potentially correlated in meaning. Additionally, bitmap files are small and can be converted into grayscale images for training the VIT model. Specifically, we collect Bitmap files from the queue, convert them into grayscale images of the same size, where each grayscale image represents a unique path within the program. We then use the VIT model to screen newly added seed files in the queue, with the smallest granularity precise to path information. We have shifted the primary task of format recognition to the VIT model, enabling more effective fuzzing test for specific formats. VERTFuzz can be adapted to a general mutation fuzz tester. We have integrated the VERTFuzz approach into AFL++, where it can function as a mode of AFL++, enabling more effective mutation of both structured and plain data.

Our combination of the VIT model and metadata tagging techniques effectively solves the problem of sparse input defect space (Zhu et al., 2022), most of the defect locations should be tagged and more targeted in subsequent mutations. Moreover, the combination of our techniques strictly limits the valid input space, which makes the mutation test samples more effective.

To assess the effectiveness and generalisability of our method, we tested it on ten different programs, including four file formats: PDF, ELF, XML, and TIFF. These formats are representative of complex file formats. The evaluation showed that compared to AFL++, our method identified 29 % more program paths and 14.54 % more program code blocks, and discovered 32 unique bugs, most of which were caused by metadata. The VIT model can filter out an average of 6.26 % of invalid test cases, and its time overhead accounts for less than 1 %, effectively improving the efficiency of subsequent fuzzing test.The main contributions of this paper are as follows:

1. We propose a fuzzing test framework called VERTFuzz, which can be combined with a dynamic structure mapping mechanism to track changes in metadata blocks in real-time, and we design a new model called ByteBERT to pinpoint the location of metadata in complex format files.
2. We have designed a new Havoc mutation strategy and proposed a weighted probability selection model based on metadata distribution features. We employed the Vision Transformer model to screen high-quality test samples and ensure the correctness of file formats.
3. We have identified 32 real-world vulnerabilities using VERTFuzz, and to facilitate future research, we have made available some of the source code at https://github.com/yaoxixixi/VERTFuzz.

2. Related work

2.1. Fuzzing for complex file parsers

With the rapid growth of software security demands, fuzzing technology has made significant advancements. Fuzzing frameworks targeting standard library functions, such as FuzzGpt, Hopper, and RPG (Deng et al., 2025; Chen et al., 2023; Xu et al., 2024), fuzzing tools for programming languages like Jazzer, Go-Fuzz, and Fuzz4all (Meumertzheim and Schneider, 2025; Vyukov, 2021; Xia et al., 2024), as well as fuzzers designed for embedded devices, such as Fuzzing BusyBox (Oliinyk et al., 2024), have demonstrated unique innovations and widespread influence within their respective application domains. However, for fuzzing test of complex file parsers, although general fuzzers such as AFL++ (A. Fioraldi et al., 2020), Honggfuzz, and OSS Fuzz can handle input data with complex file structures, most of the mutated data still cannot pass the parser's syntax check, which can result in deep level logic not being executed and reduce the possibility of discovering vulnerabilities.

Therefore, researchers proposed AFLSmart, a seed representation method that introduces advanced file structures and applies higher-order mutation operators directly to the virtual structural layers of the file, as opposed to traditional bit-level operations. This method not only maintains the legality of generated inputs but also enables deeper exploration of the input space. Furthermore, AFLSmart employs an innovative power distribution strategy based on validity, allowing it to prioritize the generation of inputs that are more likely to pass through the target program's parsing stage, thereby uncovering deeper processing logic vulnerabilities.

Weizz further developed on this basis, breaking away from the dependence on prior knowledge of input format. Weizz draw on the structural mutation concept of AFLSmart and proposes a technique for an automated structural mutation that does not require a prior input format model, focusing on handling unknown block-structured binary formats. Its method identifies the dependencies between input bytes and comparison instructions and uses dynamic metadata (such as timestamps) to infer the block structure in real time during mutation, effectively bypassing format verification barriers and enhancing the fuzzing exploration capability.

On the other hand, Steelix (Li et al., 2017) integrates static analysis techniques to filter out potentially invalid comparison operations during fuzzing effectively. Although this approach performs excellently when handling magic number tests, it still faces limitations when dealing with validation mechanisms such as checksums. T-Fuzz (Peng et al., 2018) reduces reliance on specific seeds by disabling integrity checks (e.g., field length validation) that are computationally non-critical but difficult for the fuzzer to bypass, thereby enhancing the depth of path exploration in fuzzing. Similar to TaintScope (Wang et al., 2010), this method can be combined with symbolic execution or manual analysis to address inputs that cause program crashes. Meanwhile, Driller (Stephens, 2016) introduces a dynamic switching strategy: when the fuzzer exhausts the preset budget and fails to significantly improve coverage, it automatically switches to symbolic execution mode to

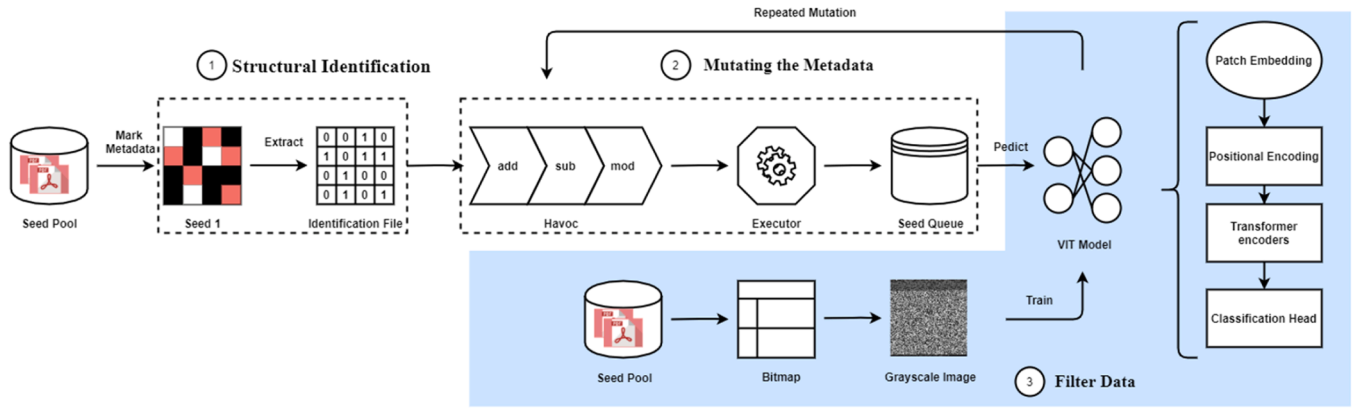

Fig. 1. Workflow of VERTFuzz.

further explore uncovered paths. Qsym (Yun et al., 2018), based on dynamic binary instrumentation, adopts a synchronized execution strategy that sacrifices speed in exchange for a more comprehensive exploration of the search space.

All of the above research works are well advanced in fuzzing test techniques for complex format files, but the existing work is still directed towards refining and identifying as many byte relationships as possible to break through the format checking and barriers. The fuzzer we designed does not strive to differentiate between different file structures, instead, we mark all file structures as metadata and feed them directly into the fuzzer for mutation, avoiding the tedious processing of input structure relationships by complex algorithms, and we take advantage of the fact that fuzzing test itself can generate a large number of mutated files in a short period of time to automate the search for direct differences in formats, and subsequently incorporate the VIT model to screen invalid test samples to ensure the correctness of fuzzing direction, which leads to stronger generalization and applicability in practical applications, with fast, automated and low-cost path coverage exploration capability.

2.2. Application of deep learning in fuzzing test

In recent years, deep learning technology, with its powerful feature extraction and pattern recognition capabilities, has gradually penetrated into many areas of software testing and security analysis (Zhang et al., 2022; Feng et al., 2023), including security testing using various large language models (Zhou et al., 2025; Zhu et al., 2025). In fuzzing, the application of deep learning primarily focuses on improving seed generation quality, optimizing mutation strategies, predicting high-value execution paths, and accelerating vulnerability detection, offering novel solutions to address the limitations of traditional methods.

Seed scheduling strategies significantly impact the coverage efficiency and crash detection capabilities of fuzzers. To address this issue, some studies have introduced reinforcement learning-based seed scheduling algorithms, which optimise seed selection order and mutation resource allocation strategies by learning from historical feedback information, thereby improving crash detection efficiency (Choi et al., 2023). RLFUZZ further models fuzzing as a Markov decision process and introduces the deep deterministic policy gradient (DDPG) algorithm to achieve continuous optimisation of scheduling strategies (Zhang et al., 2021). Additionally, some studies propose treating the fuzzing test process as a reinforcement learning challenge, using deep Q-networks (DQN) to train mutation decision strategies, effectively improving the quality of test input generation (Böttinger et al., 2018). SEAMFUZZ builds on this by introducing the Thompson sampling algorithm, customising mutation strategies based on input syntax and semantic features to achieve a more adaptive input generation mechanism (Lee et al., 2023).

To address the issue of low mutation validity in complex structured files such as PDFs, some researchers combine fuzzing test with sequence generation models, leveraging execution path modelling to guide the generation of new seeds. Some studies have proposed associating seeds with their execution paths, constructing a path modeller based on Markov chains, and using an improved sequence-to-sequence neural network (Seq2Seq RNN) to generate structurally valid PDF samples from path information and existing content, effectively activating unexplored paths (Cheng et al., 2019). Additionally, LSTM and GAN are used to model initial seed corpora, combined with the AFL fuzzifier, to achieve a learning-based seed expansion mechanism, significantly increasing the number and depth of paths (Nichols et al., 2017).

Path prediction and program behaviour modelling are key technologies for optimising input mutation strategies. NEUZZ introduces neural network control flow approximation techniques, learning the mapping relationship between paths and inputs to construct a differentiable model, and guiding input mutations through gradient calculations to enhance the efficiency and effectiveness of crash detection (She et al., 2019). SpeedNeuzz improves upon NEUZZ by proposing a method that combines gradient optimisation with advanced hashing mechanisms to simulate program behaviour, thereby further enhancing edge coverage capabilities (Li et al., 2020).

Some studies focus on the relationship between input semantics and vulnerability context, proposing semantic fuzzing test frameworks that integrate natural language processing (NLP) technology. SemFuzz extracts vulnerability-related context from unstructured text information such as CVE reports and patch logs, using semantic modelling and information extraction to assist in generating more attack-oriented test samples, thereby increasing the probability of vulnerability detection in verification paths (You et al., 2017).

A survey reveals that few researchers have used deep learning techniques to screen invalid test samples, so we innovatively incorporate deep learning techniques into the process of generating valid mutation seeds for fuzzers. Specifically, we use the VIT model to learn the bitmap file in the queue seed, and determine the relevance of the mutation seed to the program and whether it conforms to the file format, so as to filter out quality test samples.

3. Methodology

3.1. Overview

The VERTFuzz model proposes a modular decomposition framework for the fuzzing test problem of complex format files, and its core architecture consists of three key components. As shown in Fig. 1, first, the model automates the identification of the format of the input seed file by integrating the template parsing function of 010 Editor and learns the task of metadata tagging based on the ByteBERT model. In this process,

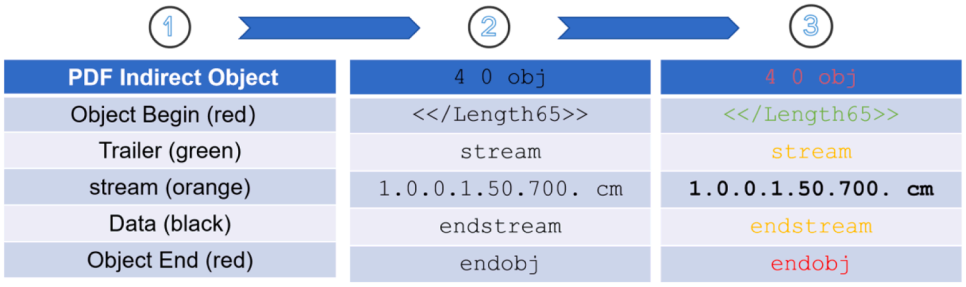

Fig. 2. PDF Indirect Object mark the color. The black area marks nonmetadata and will not be subjected to mutation operations.

the ByteBERT model generates a binary structure file with exactly the same dimensions as the original file. The structure file records metadata distribution information in the form of bitmasks, which provide precise semantic constraints for subsequent mutation operations.

In the second stage, VERTFuzz adopts a metadata-guided mutation strategy, where both the structure file and the original seed file are loaded into the testing engine. By parsing the tagged information in the structure file, the model dynamically constructs a metadata location index, enabling targeted mutations restricted exclusively to metadata regions. This approach avoids redundant modifications to user data or non-critical fields, thereby enhancing the semantic relevance and effectiveness of mutations. Furthermore, the system updates the corresponding structure files synchronously when generating new test samples, ensuring continuity and consistency in mutation operations across iterations to support persistent exploration of deep logical layers in complex formats.

The third stage introduces a Vision Transformer driven sample filtering mechanism. Upon completing initial testing, the system converts coverage bitmap data from all test cases in the execution queue into standardized grayscale images, which serve as input features for the VIT model. Through pre-training, the VIT model learns to identify global patterns (e.g., path dependencies and anomalous execution traces) in coverage bitmaps that correlate with format compliance. During inference, the model performs binary classification to filter out invalid samples with significant format deviations, retaining only structurally compliant candidates for re-entry into the testing queue. This iterative optimization mechanism dynamically adjusts sample distributions, progressively increasing the probability density of effective mutations, ultimately achieving synergistic improvements in vulnerability discovery efficiency and coverage. Experimental validation demonstrates that this strategy significantly reduces redundant testing overhead and exhibits superior generalization capability in detecting deep logical flaws in complex format parsers.

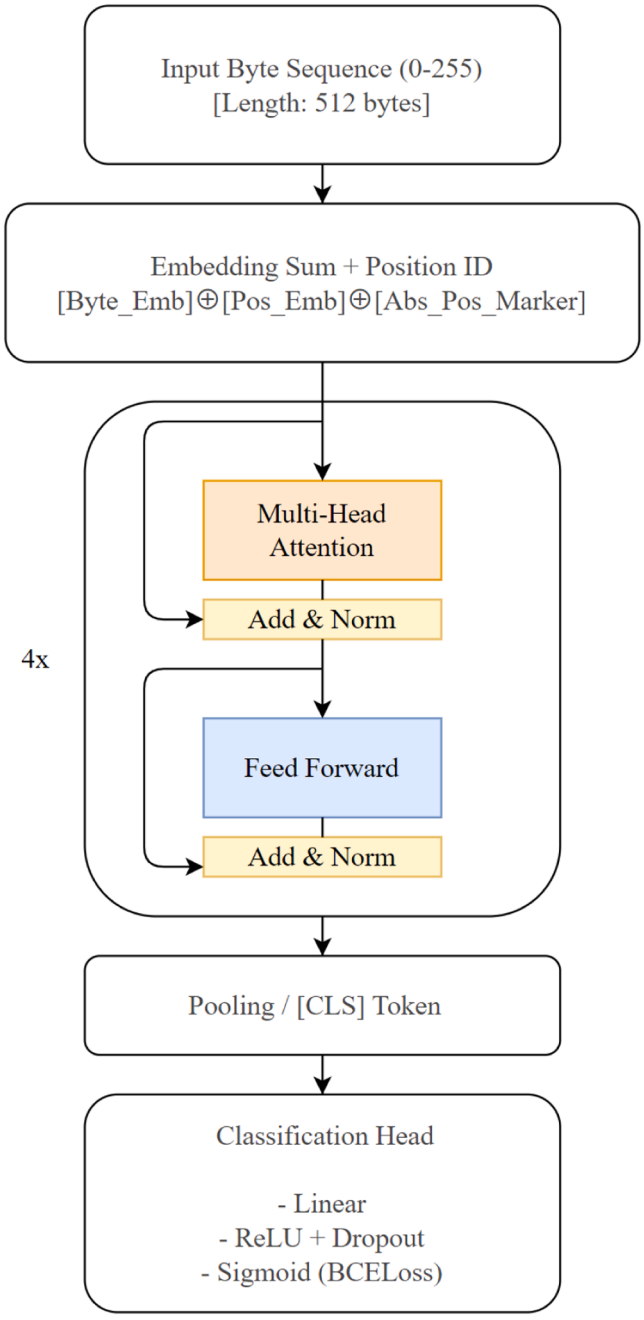

Fig. 3. ByteBERT Modeling Framework Flowchart.

3.2. *Structural identification*

010 Editor is a highly functional hexadecimal editor that provides an extensive built-in template library covering numerous common file formats. VERTFuzz introduces an automated metadata annotation framework based on 010 Editor's advanced hex editing capabilities to address challenges such as manual intervention and algorithmic redundancy in parsing complex binary file formats. 010 Editor supports syntax parsing for over 300 common file formats and offers a declarative scripting language-based template extension interface, enabling the definition of advanced logic, including conditional branches, loop controls, and function calls. Compared to traditional structured perception methods relying on symbolic execution or manual rules, this tool significantly reduces the algorithmic complexity and labor costs of format parsing.

The reason we use 010 Editor for annotation is that it explicitly reveals the semantic boundaries of a file's structure, enabling more targeted mutation. In a file with a complex format, most of the content

consists of non-metadata, while only a small portion represents metadata that defines the file structure. Typically, mutating non-metadata merely alters how content is rendered by the interpreter without affecting its core execution logic, which results in fewer discovered vulnerabilities. Allocating significant resources to this part is thus unwise. In contrast, mutations targeting metadata can significantly improve fuzzing coverage and increase the likelihood of uncovering vulnerabilities.

Within the VERTFuzz framework, 010 Editor's template engine is reconfigured as a metadata localization module. Taking PDF as an example, for the multi-layered nested structures of PDF files, the system employs customized template scripts to achieve: Regex-based syntax feature matching; Dynamic construction of a Logical Structure Tree for visualizing semantic relationships across hierarchical data units; A coloring mechanism to differentiate metadata from user data.

To enhance metadata filtering precision, the framework semantically prunes standard PDF templates. As illustrated in Fig. 2, non-metadata fields are marked as black regions, ensuring parsed results retain only core structural elements critical to format compliance. This process generates a binary metadata tagging file in bit-vector format (0/1 sequence), whose length strictly matches the original file, providing precise semantic constraint boundaries for subsequent mutation operations.

To automate the workflow, the framework implements a batch processing pipeline based on 010 Editor scripts. Firstly, the FileOpen API is used to traverse and read the collection of PDF seed files. Then, RunStyle is called to execute a customized PDF template and generate a structured parsing report. ExpandAll is used to recursively expand nested objects and construct a complete LST representation. ExportCSV is used to persistently store the positional coordinates of metadata in CSV format. Finally, a Python post-processing script is developed to map the CSV coordinates to binary files.

When using 010 Editor to mark data, there may be unknown file formats, as well as many times script files to deal with the uniform format. In order to eliminate the need to write templates, as well as to increase the versatility and scalability of the method, we designed a deep learning model based on the BERT model to automate the marking of metadata, which we named ByteBERT, as shown in Fig. 3. ByteBERT utilizes the data marked by the 010 Editor as a training set for training, and quickly tags untrained file formats by fine-tuning.

BERT model (Bidirectional Encoder Representations from Transformers) is a pre-trained language model based on the structure of Transformer (Devlin et al., 2019), which can understand the information on the left and right sides of a word through bi-directional context modeling, so as to better capture the semantic relationship. BERT adopts a "pre-training + fine-tuning" approach, and after pre-training on the large-scale corpus, it can be migrated to a variety of downstream tasks, such as question-answer, text categorization, named entity recognition, etc. Therefore, we choose the BERT model. Therefore, we select the BERT model and modify it to be more suitable for our model, so as to accomplish the task of metadata tagging.

The architecture of our model is shown in Fig. 3. It refers to the structure of the BERT model and modifies on the BERT model. ByteBERT abandons BERT's natural language oriented WordPiece segmentation strategy, directly adopts the raw byte values (0–255) as the input, and realizes the underlying binary pattern modeling through the learnable byte embedding, avoiding the need to accurately model the underlying binary pattern, and avoiding the need to accurately model the metadata. Byte Embedding is used to accurately model the underlying binary patterns, avoiding the problem of semantic fragmentation of text splitters in binary scenarios. Unlike BERT, which uses preset sinusoidal positional encoding, this model introduces Dynamic Trainable Positional Embedding and explicitly passes the absolute positional ID, which enhances the ability to capture offset-sensitive features (e.g., header and tail structure of the file) in binary files. Reduced BERT's 12-layer Transformer encoder to 4 layers, which significantly reduces computational complexity (FLOPs reduced by about 67 %) while preserving the local and global attention mechanisms, and is better suited for processing long byte sequences (window length up to 512 bytes). Eliminating the MLM/NSP pre-training phase of BERT, the end-to-end supervised learning paradigm is adopted to directly optimize the metadata classification objective through binary cross-entropy loss (BCE-Loss), achieving fast convergence with limited labeled data, and solving the data inefficiency of the pre-training-fine-tuning paradigm in the binary domain.

Algorithm 1
MUT_CLONE_COPY.

Input: temp_len, one_count, ones[]
Output: ones[],one_count

```
clone_len ← random(temp_len)
clone_from ← random(temp_len - clone_len)
diffd ← random(one_count - 1)
FOR j FROM one_count - 1 DOWNTO diffd DO
    IF j == 0 THEN
        ones[j + clone_len] ← ones[j] + clone_len
        BREAK
    ELSE
        ones[j+clone_len]← ones[j] + clone_len
    END IF
END FOR
FOR j FROM diffd TO diffd + clone_len - 1 DO
    ones[j] ← clone_len + j - diffd
END FOR
one_count ← one_count + clone_len
RETURN one_count, ones[]
```

3.3. Mutating the metadata

VERTFuzz is developed and extended based on AFL++, with a particular focus on optimizing the handling of structured information to enhance its performance in testing complex structured files. The fuzzing framework proposed in this paper takes the data identification file as input, alongside the seed files, and loads them into the fuzzing engine. During this process, a global pointer mechanism is introduced for tagging, allowing for efficient reading and processing of the file during subsequent mutation operations. In the mutation function, we introduce and maintain an array named ones, which records the indices of the seed file that correspond to metadata. This array is aligned with each round of Havoc mutation operations, ensuring consistency and accuracy of the data throughout the mutation process.

For mutations that do not alter the file length and involve only slight modifications to the file content (such as MUT_FLIPBIT, MUT_ARITH8_, MUT_RAND8, etc.), we have modified the random number generation function so that the mutation positions are determined by the content of the ones array. This ensures that each mutation operation is precisely located based on the original data, enabling fine-grained control and efficient mutation.Algorithm 1

For mutation operations that require changing the file length, such as MUT_CLONE_COPY, we first calculate the mutation positions based on the ones array and modify the data accordingly. For any newly added content, we mark and append it to the ones array, ensuring that the new data mutations maintain consistency with the previous data and preventing the omission of crucial information.

The fuzzing framework designed in this study offers the following significant advantages:

1. Efficient Data Reading and Processing: The binary identification file is only read during the initial seed mutation, where it is converted into the ones array. This approach avoids repeated file traversals, effectively reducing the computational overhead during the mutation process.

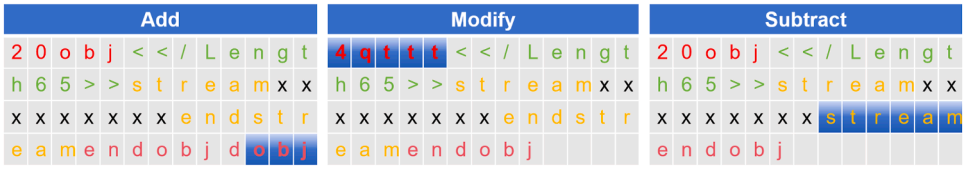

Fig. 4. The mutation process is restricted to the metadata area.

2. Fine-Grained Mutation Update Mechanism: During mutation, whether the file size changes, content is swapped, inserted, or deleted, the ones array is updated in the shortest possible time and used in subsequent mutation operations. For complex mutation operations such as MUT_SWITCH, we employ a fuzz marking strategy to ensure that all metadata mutation requirements are covered without sacrificing efficiency.
3. Optimized Memory Management and File Operations: The design avoids unnecessary heap memory allocation, maintaining only the ones array in memory. Additionally, it minimizes file read and write operations, thereby reducing time and space overhead to the maximum extent, and enhancing the overall performance and scalability of the mutation testing process.

After completing a round of havoc mutations, the generated new file will be executed through a specific function to check for the introduction of new execution paths. If a new path is discovered, the corresponding mutated test case will be added to the mutation queue. Additionally, the ones array information from the mutation process will be preserved to serve as auxiliary information in subsequent mutation operations, further enhancing mutation efficiency and coverage.

As shown in Fig. 4, during the mutation of PDF files, the mutation process is strictly confined to the marked metadata regions, excluding any unmarked non-metadata areas. Whether it is data growth, reduction, swapping, or insertion, all mutations are applied solely within the metadata structures. For example, in the mutated PDF file, the first 15 bytes will only alter the version information (e.g., %PDF-1.4), without affecting the subsequent four binary bytes, which represent unmarked binary data. Intuitively, changes to this unmarked data will not interfere with the file format's correctness. Therefore, compared to directly mutating user input data, focusing mutations on metadata can significantly enhance the file format coverage of fuzzing test, thereby improving the efficiency and depth of generating valid test cases.

3.4. Filter data

VIT is a computer vision model based on the Transformer architecture, which divides an image into fixed-size patches and inputs the embedding vectors of each patch into the Transformer model for processing. The Vision Transformer architecture enables effective processing of visual tasks such as image classification and object detection. In our fuzzing framework, VIT is used after each mutation round to filter out invalid test cases in the queue to ensure they conform to file specifications as much as possible. By invoking functions in afl-run.c, we extract the bitmap files of all test cases in the queue. The size of all bitmap files generated by the same program is consistent, and the file length is padded to become a perfect square. We then convert these bitmap files into grayscale matrix images of the same size, which are used as input test cases for the VIT model. The grayscale images consist of a large amount of binary data, with a small portion of other data representing frequently traversed code blocks, typically indicating loops or functions that are processed regularly.

We chose the VIT model to process bitmap files because each byte in a bitmap file is potentially correlated with others. Each byte in the bitmap represents a path taken in the program, and modeling long-range dependencies is crucial for determining whether the program has been correctly parsed. The VIT model is naturally appropriate for capturing global relationships, offering more direct and flexible modeling of long-distance dependencies.In addition, according to the official documentation of AFL++, the default shared memory map size is fixed. AFL++ has the ability to automatically detect the required size, and most targets use a coverage map between 20 KB and 250 KB. Based on our practical tests, the size of bitmap files usually stays below 100 KB. This relatively small file size allows each patch in the VIT model to contain fewer bytes, making the processing of each small data block more efficient. As a result, in most cases, we do not need to apply smoothing or downsampling to the image, which helps preserve important information while still achieving effective performance.Of course, some degree of information loss is inevitable. However, due to the strong interdependence between bytes, the information lost in one patch may be recoverable in another. It is precisely because of this complex correlation that we chose to integrate the VIT model into our fuzzing system.

Assuming that the obtained bitmap file has a size of $H \times W$, the original image is a grayscale image with height H and width W, denoted as. $X \in R^{H \times W}$ To adapt the bitmap for the Transformer model, we divide the bitmap into patches of size $P \times P$ Each patch is represented as $x_i \in R^{P \times P \times C}$, where C denotes the number of channels in the bitmap (1 for grayscale images). These patches are then flattened into a sequence $X_P \in R^{N \times (P^2 C)}$, where N is the total number of patches. The formula is as follows:

$$X_p = [x_1, x_2, \ldots, x_N] \tag{1}$$

Next, we use linear mapping to transform each patch into an embedding vector $e_i \in R^d$, where d is the dimensionality of the embedding space. The embedding representation of the patch is expressed as:

$$e_i = W \times x_i + b \tag{2}$$

Here, $W \in R^{(P^2 C) \times d}$represents the linear transformation matrix, and $b \in R^d$ denotes the bias term. To incorporate spatial positional information, positional encoding $P_i \in R^d$ is added to each image patch. The positional encoding is combined with the patch embeddings by summation, resulting in the weighted embedding vector z_i. The formula is as follows:

$$z_i = e_i + P_i \tag{3}$$

The positional encoding P_i can be either fixed or learnable, and it is used to preserve the relative positions of the patches within the image. In the encoder of the VIT model, the self-attention mechanism is used to model the relationships between patches. For each patch embedding z_i, we compute the attention score using the query vector $Q \in R^{N \times d}$, key vector $K \in R^{N \times d}$, and value vector $V \in R^{N \times d}$, as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V \tag{4}$$

Here, $Q = X_P W_Q$, $K = X_p W_K$, and $V = X_p W_v$ are the input matrices X_p transformed through different linear transformations to obtain the query, key, and value, respectively. W_Q, W_K, $W_v \in R^{d \times d}$ is the weight matrix. By calculating the similarity between these patches, the model is able to capture the global dependencies between the patches, thereby generating more accurate image feature representations.

After being processed by the Transformer encoder, the output of VIT is typically a feature representation that contains global information

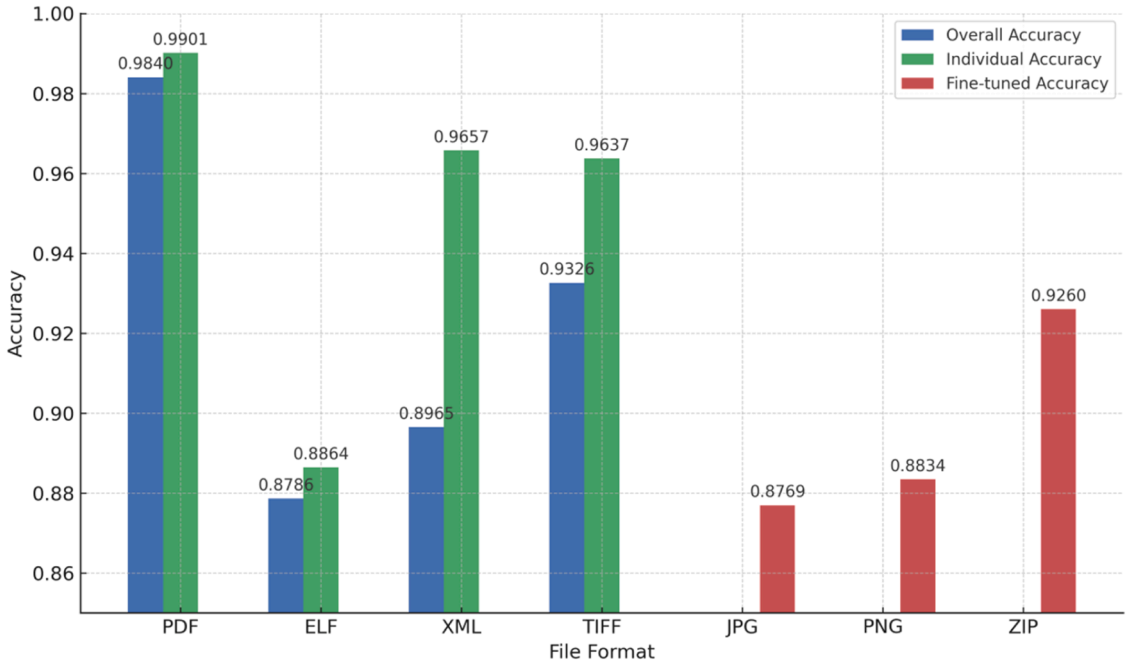

Fig. 5. Accuracy of ByteBERT model in 7 file formats.

about the image. For classification, VIT usually extracts a special [CLS] token (Wu et al., 2023) from the encoder's output and performs a linear transformation on it to obtain the final classification result. Specifically, the model computes the image's class prediction using the following formula:

$$y = \text{softmax}(W_{cls} \cdot z_{cls} + b) \tag{5}$$

Where z_{cls} is the output of the [CLS] token, $W_{cls} \in R^{d \times C}$ is the weight matrix of the classification layer, $b \in R^{C}$ is the bias term, and C is the number of classes.

In fuzzing applications, we use this classifier to determine whether the mutated bitmap files conform to the predetermined file format. Specifically, the grayscale images of the mutated samples are input into the VIT model, which then evaluates their features to decide whether they conform to the target format. The test cases that do not meet the format requirements are discarded, while the remaining test cases, in the next round of testing, will perform better.

4. Evaluation

We conducted systematic experiments to evaluate the effectiveness and efficiency of VERTFuzz in addressing the following questions:

Q1: Can our model do good seed labeling results?

Q2: How is the coverage and rate of VERTFuzz compared to other fuzzers?

Q3: Can the VIT model do a good job in screening seeds?

Q4: How does VERTFuzz perform in a real environment?

Experimental Setup: The experiments were conducted on an Ubuntu 22.04 Linux server with an 8-core Xeon(R) Gold 6240C CPU, 32GB of RAM, and an NVIDIA GeForce RTX 3080 GPU.

4.1. RQ1: seed labeling effects of the ByteBERT model

We use 010 Editor's template files to generate training sets of four formats of PDF, ELF, XML, and TIFF files, including the initial seed files and the corresponding metadata tag files. The tagging accuracy of the ByteBERT model is tested in the evaluation phase by first verifying the Individual Accuracy of whether the four file formats are tagged correctly, respectively. Then, we mixed the four formats as a training set and tested the Overall Accuracy again to evaluate the compatibility of the model with the file formats. Then we test the Fine-tuned Accuracy of JPG format, PNG format, and ZIP format by fine-tuning small samples to determine the scalability of our model. The higher the accuracy rate shown by the three experimental results, the more it proves the effectiveness of our model.

The ByteBERT model splits the input file into fixed-size chunks through a sliding window (default 512 bytes), uses byte values from 0–255 as tokens, and preserves the original location information for modeling. The model uses a lightweight BERT architecture (4 hidden layers, 4 attention heads, 128-dimensional hidden states), predicts whether each byte belongs to metadata by binary classification, and is trained using the Adam optimizer (with a learning rate of 1e-4) and the BCELoss loss function.

The experimental results are shown in Fig. 5, our ByteBERT model can cope with the task of labeling four complex format files very well, and the accuracy rate is over 87 %. PDF files perform the best, and ELF files perform poorly, this is because most of the metadata of PDF are repeated metadata, as shown in Fig. 2, while ELF files need to be indexed according to the bytes to find the next metadata, which is a more complex training task compared to PDF.

The experimental results of the model hybrid training proved that our model has a certain degree of compatibility, although compared to the individual training are reduced, but the accuracy drop remains within an acceptable range and does not significantly impact subsequent fuzzing performance. Because our fuzzers is supposed to have a certain degree of compatibility, we only need to ensure that the majority of the test content in the metadata can be. Subsequently, if we increase the sample of the training set and experimental conditions, the performance may be more outstanding.

The experimental results show that the method of testing new file formats by fine-tuning the model is feasible. We only added a small number of test files when fine-tuning the model, but the performance of the effect is still good, and the lowest JPG file format has an accuracy rate of 87.69 %. It proves that our model has some scalability.

4.2. RQ2: efficiency and coverage

We compare VERTFuzz with AFL and AFL++, both of them are

Table 1
Comparison of efficiency and coverage of VERTFuzz with other fuzzers.

File Format	Target	Fuzzer	Corpus Count	Execs Rate	Coverage	
PDF	Pdfinfo	AFL	259	**125.45**	4.68 %	+69.44 %
		AFL++	2897	67.02	6.75 %	+9.48 %
		VERTFuzz	**4292**	91.50	**7.39 %**	
	Pdftotext	AFL	1474	**119.11**	13.29 %	+46.35 %
		AFL++	6360	48.81	16.02 %	+21.41 %
		VERTFuzz	**9860**	55.39	**19.45 %**	
	Pdftopng	AFL	1358	4.09	18.02 %	+12.26 %
		AFL++	3742	4.11	18.20 %	+11.15 %
		VERTFuzz	**5270**	**9.61**	**20.23 %**	
	Pdftohtml	AFL	318	**152.20**	4.05 %	+40.24 %
		AFL++	2503	32.94	4.44 %	+27.92 %
		VERTFuzz	**3472**	37.68	**5.68 %**	
	Mupdf	AFL	1077	280.92	0.99 %	+63.63 %
		AFL++	3720	162.02	1.28 %	+26.56 %
		VERTFuzz	**5416**	**290.83**	**1.62 %**	
	Qpdf	AFL	1400	**154.59**	5.98 %	+4.68 %
		AFL++	2022	24.04	6.12 %	+2.28 %
		VERTFuzz	**2728**	38.21	**6.26 %**	
ELF	Readelf	AFL	8408	139.35	9.13 %	+126.61 %
		AFL++	13,123	133.64	18.34 %	+12.81 %
		VERTFuzz	**14,115**	**159.62**	**20.69 %**	
	Objdump	AFL	1691	**442.97**	4.13 %	+70.21 %
		AFL++	**5265**	441.20	6.28 %	+11.94 %
		VERTFuzz	3078	406.59	**7.03 %**	
XML	Libxml2	AFL	8230	1178.57	10.19 %	+24.14 %
		AFL++	10,896	**1455.05**	11.33 %	+11.65 %
		VERTFuzz	**12,354**	1311.82	**12.65 %**	
TIFF	Libtiff	AFL	3465	**996.67**	12.85 %	+86.30 %
		AFL++	7336	853.58	21.51 %	+11.29 %
		VERTFuzz	**9887**	857.93	**23.94 %**	

mutation-based fuzzers in the case of Instrumentation. Since WEIZZ fuzzer is only used in qemu (Bellard, 2005) mode, this experiment is not to be compared with this. The experiment is chosen to be on the latest version of AFL++, 4.30c, and AFL is on the latest version, 2.57bVERT-Fuzz updated on the same version of AFL++. The experiment selected a total of six under the Linux PDF tools, respectively, Xpdf-4.05 under the Pdfinfo, Pdftotext, Pdftopng, Pdftohtml, Mupdf under the Mutool-1.26.0, as well as Qpdf-11.9.1. Two under the Linux ELF tool, respectively Readefl under binutils-2.44 and Objdump. A tool to deal with XML format Libxml2–2.9.12, and a tool to deal with TIFF Libtiff-4.7.0. Each experiment is repeated 5 times, and each experiment continues to test for 24 h, and the final results are averaged. This experiment recorded the number of new edges discovered, execution rate, and coverage. Corpus Count represents the number of new paths discovered by the program, and an increase in Corpus Count indicates that new interesting test cases have been generated during the mutation process. Execs Rate represents the average number of seeds mutated per second until the end of the experiment. Coverage is the most critical metric in fuzzing test, representing the amount of code covered by all seeds. We also compared the coverage differences between VERTFuzz and two other fuzz testers to test whether our fuzz tester has a significant improvement. Higher Coverage, faster Execs Rate, and larger Corpus Count make it easier to discover vulnerabilities and also indicate higher efficiency of the fuzz tester.

According to the results in Table 1, we can find that most of the number of edges found and the coverage rate of VERTFuzz are higher than those found by the two fuzzers, AFL and AFL++, and on the Pdftohtml program, VERTFuzz has a 27.92 % improvement compared to the same version of AFL++ alone, and on the Pdftopng program, VERTFuzz also has 11.15 % higher coverage than AFL++. These results demonstrate the effectiveness of our approach. When testing the Objdump program, we find that the number of special edges found by VERTFuzz is less than the number found by AFL++, but in terms of coverage performance, VERTFuzz's coverage is higher than AFL++'s. The experimental results can show that our strategy mutates the seed into a new and interesting test sample, which covers a new major path. The AFL++ variant, on the other hand, is likely to still explore the original primary path. This validates our idea that we prefer mutating metadata over non-metadata since excessive exploration of non-metadata is not effective in discovering new coverage paths, but instead consumes the resources of the fuzz tester to some extent.

In terms of rate, the number of executions per second of VERTFuzz is mostly higher than that of AFL++, and even exceeds the rate of AFL in the four programs tested. The higher rate of AFL is a normal result of the experiments as it does not have too many queue selection and mutation algorithms compared to the other two fuzzers. The reason why our VERTFuzz has a better performance is that it has been optimized in time and space to ensure that reading binary structure files and random mutation will not case significant time loss. Secondly, during our experiments, we found that pruning the file size will not have a significant effect on the complex format files, and even the time spent on TRIM (Fioraldi et al., 2023) operation is higher than that of AFL in most of the test samples. TRIM operation took close to the time taken by the Havoc variant in most of the test samples, which was very unnecessary. So we disabled the TRIM environment variable. We also disabled the Splice (Fioraldi et al., 2023) operation, and we observed that Splice did not perform particularly well and took a longer time, probably because Splice mutates in a similar way to Havoc mutation when mutating files with complex structures.

In the face of different formats of the program, VERTFuzz can also cope with the processing, the experiment selected a total of four different formats PDF format, ELF format, XML format and TIFF format. Without observing the content of the specific formats, based on expert knowledge, we know that the PDF format is representative because the metadata and non-metadata distribution of the PDF format is always distributed multiple times alternately, and our fuzzer is better at dealing with this format compared to other fuzzer. The ELF format has the same property, followed by the XML format and TIFF format, which are also representative of other formats. We target the coverage of these four formats were calculated to improve the average number of experimental results, as described above, PDF and ELF have the best experimental results.

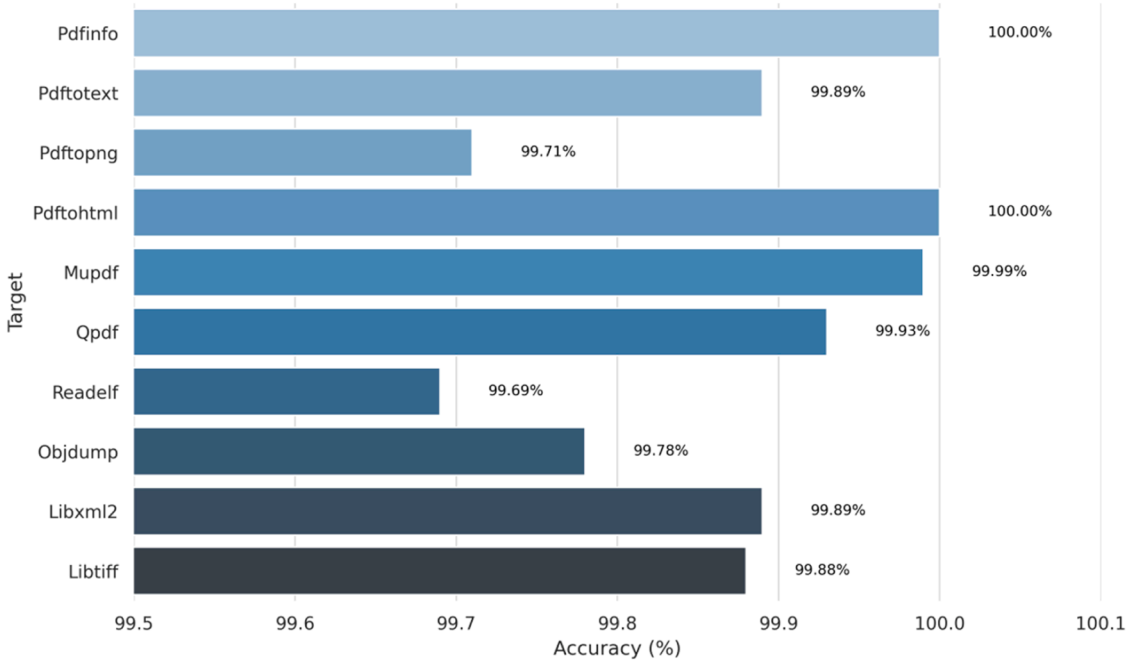

Fig. 6. The VIT model filters the accuracy of each program format.

Overall, our method found 29.00 % more program paths and 14.54 % more program code blocks. The coverage and rate of VERTFuzz compared to the other two fuzzers have a certain degree of improvement in the balance between the rate and the effectiveness of VERTFuzz to do a better job.

4.3. RQ3: the performance of the VIT model

At the end of each round of experiments, we extracted all the test samples in the VERTFuzz queue, extracted the corresponding bitmap graphs, and converted them to grayscale graphs of the same size after padding to serve as the test set for the VIT model. We select the training files that match the format of the program under test and other formats, and use the corresponding grayscale maps for binary classification training under the same conditions. The trained model has the ability to determine whether the test samples experience the path according to the normal specification of the program. We tested the accuracy of 10 different programs in screening invalid seeds under the VIT model to verify whether our model has the ability to determine file formats.

The training results are shown in Fig. 6. The training process occurs before the fuzzer, so it will not affect the efficiency of the fuzzer. For different programs, different models need to be trained, even for the same file format, the bitmap graphs of different programs will be different, a total of ten models were trained in this experiment, and it can be found that the accuracy rate is almost all close to 100 %, so we think that it is easy to judge whether a test sample conforms to the file format of a program. The lowest correctness of the experimental results is 99.69 %, which has a minimal impact on the subsequent judgment of the test samples in the queue, because for the fuzzer, adding or subtracting a similar seed does not have much impact on the final result of the fuzzing test.

Since the time consumed and the number of completed rounds vary when testing different programs, we counted the number of test samples each program has in its queue, the number of invalid samples screened out by the model, and the model overhead time over a 24-hour period, and calculated the percentage of invalid test samples screened out, as well as the percentage of the model overhead time. The model overhead time includes the time required by the model itself for prediction, the time taken by the test samples to generate the grayscale map, and the time spent by the fuzzer to interact with the model. The number of invalid samples screened out by the model is used to verify the model's contribution to VERTFuzz, and the model's overhead time is calculated to prove its effectiveness.

Table 2
The number of test samples filtered by the VIT model and the time overhead of the VIT model.

Target	Non compliant test samples	Filter percentage	time consumption	Time percentage
Pdfinfo	342	7.96 %	270.40	0.31 %
Pdftotext	281	2.84 %	612.78	0.70 %
Pdftopng	205	3.88 %	331.52	0.38 %
Pdftohtml	47	1.35 %	215.57	0.25 %
Mupdf	42	0.77 %	357.19	0.41 %
Qpdf	234	8.57 %	173.21	0.20 %
Readelf	2364	16.74 %	889.24	1.02 %
Objdump	125	4.06 %	200.07	0.23 %
Libxml2	985	7.97 %	827.71	0.95 %
Libtiff	841	8.50 %	622.88	0.72 %

As shown in the Table 2. After 24 h of fuzzing test, we found that our VIT model can screen 6.26 % of invalid test samples on average. After sampling, we found that when the accuracy of setting test samples is above 99 %, the format of the document is basically correct, on the contrary, if the accuracy is less than 99 %, the PDF parser will show that this PDF file has the wrong format. However, from the experimental results, the difference between the two is not large, you can choose a different threshold according to the actual situation. In experiments, we use 99 % accuracy, it can be observed that in the test Readelf program, it is best to have 16.74 % of the invalid test samples filtered. This improves the accuracy of the fuzzing test in later rounds of testing.

The model overhead time is mainly related to the number of new edges generated by the fuzzing test process, as shown in Table 2. It can be observed that the number of edges generated by each program, a high number of edges means that there are more test samples in the queue, and the higher the time overhead of the VIT model filtering, we calculated that the overhead time of all the programs for each test sample is about 0.065 s, which means that the VIT model filtering one test sample takes 0.065 s. We calculated that all programs generate an average of

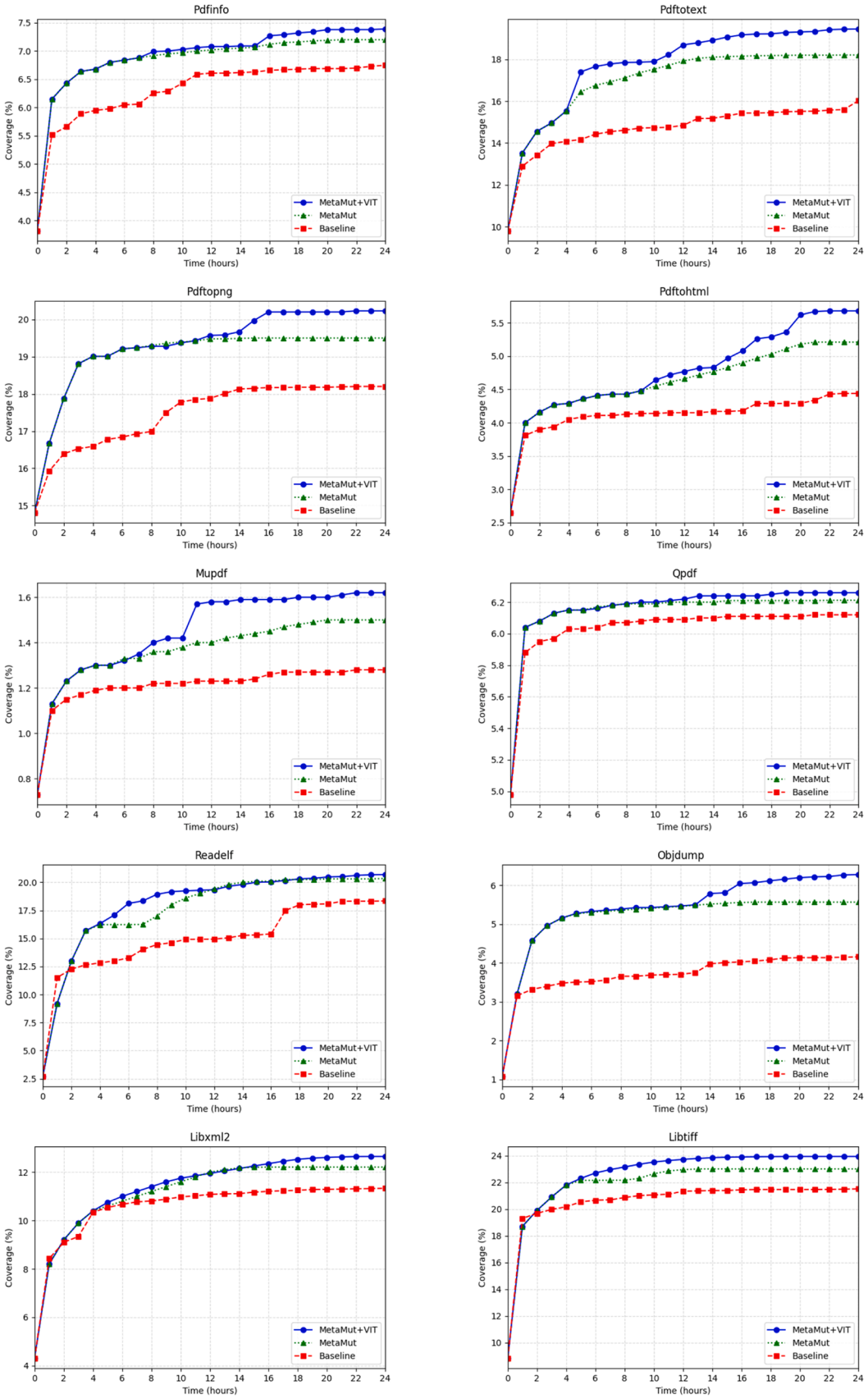

Fig. 7. Changes in coverage over 24 h for three groups of comparison experiments.

325.91 new variant files per second, which means that for every model screened by VIT, 21 fewer files are varied, which we consider to be a very small overhead because we do not screen every variant file, and the percentage of test samples that are screened by VIT is not very large. We also counted the overhead percentage of the VIT model over a 24-hour period, and found that the highest is around 1 %, proving that our model time overhead is tiny.

We continued to test the contribution of the VIT model to the entire fuzz tester, especially whether the addition of VIT would improve the efficiency of VERTFuzz. Therefore, we designed the following ablation experiment. Under constant experimental conditions, this study designed three groups of comparison experiments by the control variable method: the first group (Baseline) used AFL++ as the benchmark testing framework; the second group (MetaMut) added optimization for metadata variation on the basis of the first group; and the third group (MetaMut + VIT) further integrated the VIT model screening mechanism. The experiment continuously observes the trend of code coverage changes of the three groups of fuzzing test tools within 24 h.

The results are shown in Fig. 7. According to the experimental results, we find that the coverage performance of the third group is better than the coverage performance of the first two groups. It proves that combining the VIT model and metadata-guided mutation can indeed be superior, and from the ten groups of experiments in the figure, we can find that the final coverage result of the second group is biased towards the coverage result of the third group, indicating that metadata variation is more dominant because it affects the coverage change through the variation strategy, whereas the VIT model affects the variation result by influencing the subsequent seeds.

Overall, our tests verified that the VIT model has high prediction accuracy and can screen out most invalid seeds. At the same time, the VIT model has low time overhead, and experiments have proven its contribution to VERTFuzz. Various experimental data show that the VIT model performs exceptionally well.

4.4. RQ4: real-world performance

We ran our fuzz tester in a real-world environment for 72 h and identified a total of 39 bugs, seven of which were also detected by AFL++ under the same experimental conditions. Additionally, 32 unique vulnerabilities were identified by VERTFuzz: Objdump (3 bugs), Readelf (1 bug), Pdfinfo (5 bugs), Pdftotext (13 bugs), Pdftopng (7 bugs), Cpdf (1 bug), and Libxml2 (2 bugs). Among these, 13 vulnerabilities were newly discovered, and one CVE identifier has been assigned. Most of these vulnerabilities involve one-byte stack overflows, integer overflows, and structure loop issues. We will analyse two unique examples below.

CVE-2024–54,731. This is the first CVE number we applied for, and it is a stack vulnerability in CPDF v2.8. Cpdf up to version 2.8 allows stack overflows through constructed PDF documents. Essentially, it is a structure loop vulnerability where the Pages object references itself, causing a structure loop. The parser may enter infinite recursion, ultimately leading to a stack overflow. We extracted the vulnerability samples we discovered. To trigger this vulnerability, two conditions must be met simultaneously in the PDF: the presence of another object's ID in two objects, and all these data belong to metadata. This is relatively challenging for other fuzz testers, as AFL++'s mutation space covers the entire file, whereas our fuzz tester can narrow the input space, making mutations more effective.

CVE-2024–7867. The vulnerability discovered by our fuzz tester aligns with the content of this CVE number, though the trigger location differs. This vulnerability arises because page boxes (MediaBox, CropBox, etc.) with extremely large coordinates may trigger integer overflow and division by zero. This is an interesting vulnerability; simply changing '/MediaBox [0 0 42 99,213]' to '/MediaBox [0 0 4255,555,555 99,213]' can trigger the vulnerability. However, AFL++'s performance in this scenario remains suboptimal, as the seed required to trigger this vulnerability has a large file size, making the probability of achieving the desired mutation on a number very low. Our fuzz tester addresses the issue of sparse input defect space, thereby accelerating the discovery of such vulnerabilities.

5. Discussion

Although our work has achieved some progress and success, there are still certain limitations. In previous discussions, we learned from official documents that the size of bitmap files typically ranges between 20–250 KB. However, some programs still exceed this value significantly. The largest known bitmap file comes from a program in LibreOffice, with a size of 5 MB. Our model requires higher hardware specifications when processing such large data.

In future work, we will continue to improve the VIT model to accommodate larger data inputs. We will compare it with deeper learning models that have performed better in recent years and integrate them into our fuzzing test framework. We may also introduce large language models to replace the VIT model. We also envision applying deep learning techniques to mutation scheduling, path analysis, and behaviour prediction. By combining execution optimisation techniques, we aim to allocate most of the fuzz tester's resources to analysing primary paths and mutations. We have planned several scoring criteria and intend to align them with the work described in this paper to enhance the performance of our fuzz tester.

6. Conclusion

In this paper, we propose VERTFuzz, a fuzzing test framework for complex format documents based on metadata tagging and the Visual Transformer model. By combining the template parsing capability of 010 Editor with the ByteBERT model based on deep learning, we realize the automated metadata tagging and accurate localization of complex format documents such as PDF, and on this basis, The mutation strategy of AFL++ is reconstructed, and a weighted probabilistic selection model based on metadata distribution features is proposed. Experimental results demonstrate that VERTFuzz significantly improves both code coverage and execution efficiency compared to existing general-purpose fuzzing frameworks. On average, 29 % more program paths and 14.54 % more code blocks are found, and the execution rate outperforms AFL++ in some scenarios. In addition, by introducing the VIT model for path-level filtering of the test queue, VERTFuzz is able to effectively filter 6.26 % of invalid test cases with less time overhead. In real environment tests, VERTFuzz successfully discovered 32 unique vulnerabilities, validating its effectiveness in complex logic defect detection.

CRediT authorship contribution statement

Zhaoyu Wen: Writing – original draft. **Zhiqiang Wang:** Writing – review & editing. **Biao Liu:** Writing – review & editing.

Declaration of competing interest

I have The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

Bellard, F., 2005. QEMU, a fast and portable dynamic translator. In: Proc. USENIX Annu. Tech. Conf., FREENIX Track, 41, pp. 10–5555.

Böttinger, K., Godefroid, P., Singh, R., 2018. Deep reinforcement fuzzing. 2018 IEEE Security and Privacy Workshops (SPW). San Francisco, CA, USA, pp. 116–122. https://doi.org/10.1109/SPW.2018.00026.

Chen, P., Xie, Y., Lyu, Y., 2023. Hopper: interpretative fuzzing for libraries. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, pp. 1600–1614.

Cheng, L., et al., 2019. Optimizing seed inputs in fuzzing with machine learning. In: 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion). Montreal, QC, Canada, pp. 244–245. https://doi.org/10.1109/ICSE-Companion.2019.00096.

Choi, G., Jeon, S., Cho, J., Moon, J., 2023. A seed scheduling method with a reinforcement learning for a coverage guided fuzzing. IEEE Access 11, 2048–2057. https://doi.org/10.1109/ACCESS.2022.3233875.

Y. Deng, C.S. Xia, C. Yang, S.D. Zhang, S. Yang, L. Zhang, 2025. Large language models are edge-case fuzzers: testing deep learning libraries via fuzzgpt, arXiv preprint arXiv:2304.02014.

Devlin, J., Chang, M.W., Lee, K., et al., 2019. Bert: pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, 1, pp. 4171–4186 (long and short papers).

A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, 2025. An image is worth 16x16 words: transformers for image recognition at scale, arXiv preprint arXiv:2010.11929.

Feng, X., Zhu, X., Han, Q.-L., Zhou, W., Wen, S., Xiang, Y., 2023. Detecting vulnerability on IoT device firmware: a survey. IEEE/CAA J. Autom. Sin. 10 (1), 25–41. https://doi.org/10.1109/JAS.2022.105860. January.

Fioraldi, A., D'Elia, D.C., Coppa, E., 2020. WEIZZ: automatic grey-box fuzzing for structured binary formats. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 1–13.

Fioraldi, A., Maier, D., Eißfeldt, H., 2020. {AFL++}: combining incremental steps of fuzzing research. In: Proc. 14th USENIX Workshop on Offensive Technologies (WOOT 20).

Fioraldi, A., Mantovani, A., Maier, D., 2023. Dissecting American Fuzzy Lop: a FuzzBench evaluation. ACM Trans. Softw. Eng. Methodol 32 (2), 1–26.

Lee, M., Cha, S., Oh, H., 2023. Learning seed-adaptive mutation strategies for greybox fuzzing. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). Melbourne, Australia, pp. 384–396. https://doi.org/10.1109/ICSE48619.2023.00043.

Li, Y., Chen, B., Chandramohan, M., Lin, S.-W., Liu, Y., Tiu, A., 2017. Steelix: program-state based binary fuzzing. In: Proc. 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2017), pp. 627–637. https://doi.org/10.1145/3106237.3106295.

Li, Y., Xiao, X., Zhu, X., Chen, X., Wen, S., Zhang, B., 2020. SpeedNeuzz: speed up neural program approximation with neighbor edge knowledge. In: 2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom). Guangzhou, China, pp. 450–457. https://doi.org/10.1109/TrustCom50675.2020.00068.

Meumertzheim, F., Schneider, N., 2025. jazzer. https://github.com/CodeIntelligenceTesting/jazzer (accessed 01 March 2025).

N. Nichols, M. Raugas, R. Jasper, Faster fuzzing: reinitialization with deep neural models. arXiv preprint arXiv:1711.02807, 2017.

Oliinyk, Y., Scott, M., Tsang, R., 2024. Fuzzing BusyBox: leveraging LLM and crash reuse for embedded bug unearthing. 33rd USENIX Secur. Symp. (USENIX Secur. 24) 883–900.

Peach Teach, 2020. Peach, Peach Fuzzer. http://www.peachfuzzer.com/ (accessed 01 March 2025).

Peng, H., Shoshitaishvili, Y., Payer, M., 2018. T-fuzz: fuzzing by program transformation. In: 2018 IEEE Symposium on Security and Privacy (SP). CA, USA. San Francisco, pp. 697–710. https://doi.org/10.1109/SP.2018.00056.

Pham, V.-T., Böhme, M., Santosa, A.E., Căciulescu, A.R., 2021. A. Roychoudhury, Smart Greybox fuzzing. IEEE Trans. Softw. Eng. 1980–1997. https://doi.org/10.1109/TSE.2019.2941681.

robertswiecki, R., Bechtsoudis, A., 2024. Honggfuzz. https://github.com/google/honggfuzz (accessed 01 March 2025).

Serebryany, K., 2017. OSS-Fuzz — Google's continuous Fuzzing Service For Open Source Software. USENIX Security, pp. 131–142.

She, D., Pei, K., Epstein, D., Yang, J., Ray, B., Jana, S., 2019. NEUZZ: efficient fuzzing with neural program smoothing. In: 2019 IEEE Symposium on Security and Privacy (SP). CA, USA. San Francisco, pp. 803–817. https://doi.org/10.1109/SP.2019.00052.

Stephens, N., 2016. Driller: augmenting fuzzing through selective symbolic execution. In: Proc. 23rd Annu. Netw. Distrib. Syst. Secur. Symp, pp. 1–16.

SweetSweet, L., raeme, G., 2024. 010 Editor. https://www.sweetscape.com/010editor/ (accessed 03 March 2025).

Vyukov, D., 2021. go-fuzz. https://github.com/dvyukov/go-fuzz (accessed 01 March 2025).

Wang, T., Wei, T., Gu, G., Zou, W., 2010. TaintScope: a checksum-Aware directed fuzzing tool for automatic software vulnerability detection. In: 2010 IEEE Symposium on Security and Privacy. CA, USA. Oakland, pp. 497–512. https://doi.org/10.1109/SP.2010.37.

L. Wu, W. Zhang, T. Jiang, [CLS] Token is all you need for zero-shot semantic segmentation, arXiv preprint arXiv:2304.06212, 2023.

Xia, C.S., Paltenghi, M., Tian, J.L., Pradel, M., Zhang, L., 2024. Fuzz4ALL: universal fuzzing with large language models. In: 2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE). Lisbon, Portugal, pp. 1547–1559.

Xu, Z., Wu, B., Wen, C., Zhang, B., Qin, S., He, M., 2024. RPG: rust library fuzzing with pool-based fuzz target generation and generic support. In: 2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE). Lisbon, Portugal, pp. 1521–1533. https://doi.org/10.1145/3597503.3639102.

You, W., Zong, P., Chen, K., et al., 2017. Semfuzz: semantics-based automatic generation of proof-of-concept exploits. In: Proceedings of the 2017 ACM SIGSAC conference on computer and communications security, pp. 2139–2154.

Yun, I., Lee, S., Xu, M., Jang, Y., Kim, T., 2018. QSYM: a practical concolic execution engine tailored for hybrid fuzzing. USENIX Secur. Symp. ser. USENIX 745–761.

Zalewski, M., Moroz, M., Mishra, D., 2020. American fuzzy lop. https://github.com/google/AFL (accessed 03 March 2025).

Zhang, J., Pan, L., Han, Q.-L., Chen, C., Wen, S., Xiang, Y., 2022. Deep learning based attack detection for cyber-physical system cybersecurity: a survey. IEEE/CAA J. Autom. Sin. 9 (3), 377–391. https://doi.org/10.1109/JAS.2021.1004261. March.

Zhang, Z., Cui, B., Chen, C., 2021. Reinforcement learning-based fuzzing technology. In: Innovative Mobile and Internet Services in Ubiquitous Computing: Proceedings of the 14th International Conference on Innovative Mobile and Internet Services in Ubiquitous Computing (IMIS-2020), pp. 244–253.

Zhou, W., et al., 2025. The security of using large language models: a survey with emphasis on ChatGPT. IEEE/CAA J. Autom. Sin. 12 (1), 1–26. https://doi.org/10.1109/JAS.2024.124983. January.

Zhu, X., Wen, S., Camtepe, S., Xiang, Y., 2022. Fuzzing: a survey for roadmap. ACM Comput Surv 1–36. https://doi.org/10.1145/3512345.

Zhu, X., Zhou, W., Han, Q.-L., Ma, W., Wen, S., Xiang, Y., 2025. When software security meets large language models: a survey. In: IEEE/CAA Journal of Automatica Sinica, 12, pp. 317–334. https://doi.org/10.1109/JAS.2024.124971. February.